FLIGHT DELAY PREDICTION USING MACHINE LEARNING

Bachelors of Technology in Computer Science Engineering

By — Paras Dhatwalia (17232990042)

UNIVERSITY INSTITUTE OF TECHNOLOGY, SHIMLA

TABLE OF CONTENTS

1. Introduction

2. Background Study

3. Problem Statement

4. Objectives

5. Literature Review

6. Dataset Description

7. Data Preprocessing

8. Temporal Train–Validation–Test Splitting

9. Exploratory Data Analysis

10. Feature Engineering

11. Machine Learning Algorithm — LightGBM

12. Hyperparameter Optimisation with Optuna

13. Model Training Process

14. Model Evaluation Metrics

15. Results and Analysis

16. SHAP Interpretability and Leakage Audit

17. Confusion Matrix Analysis

18. Full-Stack Web Application

19. Docker Deployment

20. Project Structure

21. Design Decisions and Trade-offs

22. Limitations

23. Future Scope

24. Conclusion

25. Tools and Technologies

26. References

ABSTRACT

Flight delays impose significant economic and operational costs on airlines, airports, and passengers worldwide. While airlines monitor delays internally, passengers and operational planners lack accessible, real-time predictive tools. This project builds an end-to-end machine learning system that predicts whether a commercial flight will arrive 15 or more minutes late, using only information available 24 hours before scheduled departure.

The system employs a LightGBM gradient-boosted decision tree trained on the U.S. Bureau of Transportation Statistics (BTS) On-Time Performance dataset. A 17-feature engineering pipeline transforms raw flight records through cyclic temporal encoding, Haversine great-circle distance computation, K-Fold smoothed target encoding, historical delay aggregates, and holiday proximity detection. A strict temporal split (January–August for training, September–October for validation, November–December for testing) simulates real deployment conditions where the model only ever predicts genuinely future flights.

Evaluation uses Precision–Recall AUC (PR-AUC) as the primary metric—more informative than ROC-AUC given the ~20% imbalanced positive class. SHAP TreeExplainer provides per-prediction feature attributions, enabling interpretable risk explanations. A comprehensive three-stage leakage audit (blacklist enforcement, high-correlation scan, and SHAP importance sanity check) ensures that no post-departure information contaminates the predictions.

The trained model is deployed as a production-ready web application: a FastAPI backend serves predictions with SHAP-based explanations via a REST API, and a React + Tailwind CSS frontend presents an interactive flight lookup interface with a probability gauge, risk-factor bar chart, and dark-mode support. Both services are containerised with Docker Compose for reproducible, single-command deployment. This project demonstrates that principled machine learning—combining rigorous leakage prevention, appropriate evaluation metrics, model interpretability, and modern

full-stack engineering—can produce actionable predictive tools for transportation logistics.

1. Introduction

Flight delays are one of the most persistent operational challenges in commercial aviation. According to the U.S. Federal Aviation Administration (FAA), flight delays cost the U.S. economy over $30 billion annually, encompassing airline operational costs, passenger lost productivity, and downstream cascading effects across airport networks. In 2023, approximately 21% of U.S. domestic flights arrived more than 15 minutes behind schedule, affecting hundreds of millions of passengers.

The causes of flight delays are multifaceted: weather disruptions, air traffic congestion, mechanical issues, crew scheduling conflicts, and airport capacity constraints all contribute. While airlines maintain internal operations centres that monitor delays, the travelling public and third-party operational planners have limited access to predictive delay information. Existing consumer-facing tools (e.g., FlightAware, Google Flights) primarily report real-time status rather than pre-departure predictions.

From a machine learning perspective, flight delay prediction is a compelling problem. It features a moderately imbalanced binary target (~20% delayed), rich categorical features (airline, airport codes, aircraft tail numbers), temporal patterns (hour-of-day, day-of-week, seasonal effects), and geospatial structure (route distances). The problem also introduces a critical constraint not found in typical classification tasks: **temporal information leakage**. Many columns in flight performance datasets—such as taxi times, actual departure and arrival times, and carrier delay breakdowns—are only available *after* a flight has departed. Using these features would produce artificially inflated accuracy that is entirely useless in practice.

This project addresses this challenge by building a complete machine learning pipeline with a strict T-24h (24 hours before departure) information boundary. Every feature used by the model—from cyclic departure-time encoding to smoothed historical airline delay rates—is knowable a full day before the flight departs. The system spans the full ML lifecycle: data ingestion, temporal splitting, feature engineering, model training with Bayesian hyperparameter optimisation, evaluation with SHAP interpretability, and deployment as a full-stack web application with Docker containerisation.

2. Background Study

2.1 The Flight Delay Problem—Operational Context

Commercial aviation operates on tightly coupled networks. A single delayed departure at a hub airport can cascade through connecting flights, crew rotations, and gate assignments, creating system-wide disruptions. The Bureau of Transportation Statistics (BTS) tracks on-time performance for all U.S. domestic flights operated by carriers with at least 0.5% of domestic scheduled-service passenger revenue.

BTS defines a flight as "delayed" when it arrives 15 or more minutes after its scheduled arrival time. This 15-minute threshold is the industry standard used by the Department of Transportation (DOT) for regulatory reporting and forms the basis for contractual obligations between airlines and airports.

Delay causes are classified into five categories: *carrier delays* (maintenance, crew, aircraft cleaning), *weather delays*, *National Aviation System (NAS) delays* (air traffic control, airport operations), *security delays*, and *late aircraft delays* (propagation from a previous flight). Notably, these causal breakdowns are only available post-flight and must not be used for prediction.

2.2 Traditional Approaches and Their Limitations

Traditional delay prediction relies on operations research methods: queuing models for airport congestion, network flow models for propagation effects, and statistical regression on historical averages. Airlines use proprietary systems that combine real-time weather feeds, air traffic flow management data, and crew scheduling information.

These approaches have significant limitations for general-purpose prediction. Queuing models require real-time airport state information that is unavailable 24 hours in advance. Statistical averages fail to capture non-linear interactions between factors (e.g., a particular airline at a particular airport during holiday season). Proprietary airline systems are not accessible to passengers or third-party planners.

2.3 Machine Learning for Flight Delay Prediction

Machine learning offers a data-driven alternative that can capture complex, non-linear relationships from historical flight records. Gradient-boosted decision trees (GBDT), particularly implementations such as LightGBM and XGBoost, have emerged as the dominant approach for tabular prediction tasks. They handle mixed feature types natively, are robust to feature scaling, provide built-in feature importance, and train efficiently on datasets with millions of records.

A critical challenge unique to this domain is **information leakage**. Flight performance datasets contain both pre-departure features (scheduled times, airline, airports) and post-departure features (actual times, taxi durations, delay breakdowns). Naïve inclusion of post-departure features produces models with artificially high accuracy (>90%) that are entirely useless in practice because those features are unknown at prediction time. Rigorous feature blacklisting and leakage auditing are therefore essential.

2.4 SHAP Interpretability

Shapley Additive exPlanations (SHAP), introduced by Lundberg and Lee (2017), provide a theoretically grounded framework for explaining individual predictions. SHAP values decompose each prediction into additive contributions from each feature, satisfying desirable axiomatic properties: local accuracy, missingness, and consistency. For tree-based models, the TreeSHAP algorithm computes exact Shapley values in polynomial time, making it practical for production use.

In the flight delay context, SHAP enables actionable explanations such as: “This flight has a 72% delay probability primarily because the airline’s historical delay rate is high (+15%) and the departure time is during peak evening hours (+12%).”

3. Problem Statement

Despite extensive data collection by the Bureau of Transportation Statistics, extracting actionable pre-departure delay predictions remains challenging. Existing prediction systems either require real-time operational data unavailable to the public, or fail to enforce the temporal boundary between pre- and post-departure information, thereby producing misleading performance estimates.

This project addresses the following problem:

> ***Given only information available 24 hours before a scheduled departure—airline, origin airport, destination airport, scheduled departure time, and aircraft tail number—can a machine learning model accurately predict whether the flight will arrive 15 or more minutes late?***

The task is formulated as a supervised binary classification problem:

- **Label:** `y = 1` if arrival delay ≥ 15 minutes; `y = 0` otherwise
- **Positive class (~20%):** Delayed flights (minority class)
- **Negative class (~80%):** On-time flights (majority class)

Beyond raw accuracy, the model must satisfy four domain-specific requirements:

1. **Strict temporal leakage prevention:** A blacklist of 14 post-departure columns (26 including case variants) is enforced at data loading time. A SHAP-based sanity check further verifies that no single feature contributes more than 40% of total importance.
2. **Appropriate evaluation for class imbalance:** With a ~20% positive rate, accuracy alone is misleading (a naïve "always on-time" classifier achieves ~80%). PR-AUC, which focuses on the minority class, serves as the primary evaluation metric.
3. **Interpretable predictions:** Each prediction must include the top contributing factors with signed SHAP impact values, enabling users to understand *why* a flight is predicted to be delayed.
4. **Temporal evaluation integrity:** Training, validation, and test sets must be split chronologically—not randomly—to simulate real deployment where the model predicts

future flights based on past data only.

4. Objectives

Primary Objective

To develop, evaluate, and deploy a machine learning binary classification system that predicts flight arrival delays ≥15 minutes using only pre-departure features, achieving a PR-AUC greater than 0.35 on a temporally held-out test set, with zero data leakage and per-prediction SHAP explanations.

Secondary Objectives

1. **Data Pipeline:** Build a robust data loading module that enforces a blacklist of 14 post-departure columns (26 including case variants), preventing temporal information leakage at the earliest stage.
2. **Temporal Splitting:** Implement a calendar-based chronological split (Jan–Aug / Sep–Oct / Nov–Dec) with assertion guards that raise an error if date ranges overlap.
3. **Feature Engineering:** Design a 17-feature pipeline comprising cyclic sin/cos encoding, Haversine great-circle distance (using 139 airport coordinates), K-Fold smoothed target encoding (λ=20), historical delay rate aggregates (with minimum-sample thresholds of 30 and 100), and US federal holiday proximity flags (±2 days).
4. **Model Training:** Train a LightGBM gradient-boosted decision tree with up to 3,000 boosting rounds, early stopping (patience = 100), and optional class-weight balancing.
5. **Hyperparameter Optimisation:** Implement Optuna-based Bayesian optimisation (TPE sampler) over 9 hyperparameters with 50 trials, maximising validation PR-AUC.
6. **Comprehensive Evaluation:** Report PR-AUC, ROC-AUC, F1-score, precision at 50% recall, recall at 80% precision, Brier score, and confusion matrix on the held-out November–December test set.
7. **SHAP Interpretability:** Generate global feature importance (beeswarm and bar plots), per-prediction top-5 factor explanations, and partial dependence plots for the top 3 features.
8. **Leakage Audit:** Implement a three-stage audit—blacklist check, high-correlation scan ($|r| > 0.95$ threshold), and SHAP importance sanity check (>40% single-feature threshold)—run on every training cycle.

9. **REST API:** Deploy the model as a FastAPI backend with `POST /predict` and `GET /health` endpoints, Pydantic v2 request validation, CORS support, and request-logging middleware.
10. **Interactive Frontend:** Build a React 18 + Vite + Tailwind CSS single-page application with a flight input form, probability gauge meter (Recharts), SHAP factor chart, loading skeleton, error toast, and dark mode.
11. **Docker Deployment:** Containerise both backend and frontend with Docker Compose, including health checks, read-only artifact volumes, and a shared bridge network.

5. Literature Review

Research on flight delay prediction has evolved significantly over the past two decades, progressing from simple statistical models to sophisticated ensemble methods and interpretable AI.

Rebollo and Balakrishnan (2014) applied Random Forests to predict departure delays at major U.S. airports, demonstrating that tree-based ensembles outperform linear models by capturing non-linear interactions between weather, traffic, and temporal features. Their work highlighted the importance of temporal features (hour-of-day, day-of-week) as primary predictors of delay.

Mueller and Chatterji (2002) established the foundational understanding of delay propagation in air traffic networks, showing that delays cascade through connected flights in a hub-and-spoke system. This insight motivated the inclusion of historical route-level and airline-level delay rates as features in the present work.

Kim et al. (2016) compared multiple classification algorithms—Logistic Regression, SVM, Random Forest, and Gradient Boosting—on BTS data and found that Gradient Boosting consistently achieved the highest AUC scores. Critically, their work also emphasised the importance of removing post-departure features to avoid information leakage.

Ke et al. (2017) introduced LightGBM, a gradient-boosting framework that uses histogram-based splitting and leaf-wise tree growth. LightGBM achieves state-of-the-art performance on tabular data while training 10–20× faster than traditional GBDT implementations, making it practical for large-scale flight datasets containing millions of records.

Akiba et al. (2019) introduced Optuna, a hyperparameter optimisation framework using Tree-structured Parzen Estimators (TPE). TPE models the conditional probability of promising hyperparameter configurations and samples preferentially from high-performing regions, converging to near-optimal configurations in 30–50 trials on typical 10-dimensional search spaces.

Lundberg and Lee (2017) introduced SHAP (SHapley Additive exPlanations), providing a unified framework for feature attribution grounded in cooperative game theory. For tree-

based models, their TreeSHAP algorithm computes exact Shapley values in polynomial time, enabling per-prediction explanations at production speed.

Choi et al. (2016) demonstrated that feature engineering—specifically cyclic encoding of temporal variables—significantly improves delay prediction accuracy. Cyclic sin/cos encoding preserves the circular nature of time (i.e., 23:59 is temporally adjacent to 00:01), which standard ordinal encoding misrepresents.

Hensman and Masko (2015) investigated temporal train/test splitting for time-dependent prediction tasks, demonstrating that random shuffled splits produce optimistically biased performance estimates. For flight delay prediction, where patterns evolve seasonally and are influenced by non-stationary factors, temporal splitting is essential for honest model evaluation.

6. Dataset Description

6.1 Data Source

The dataset used is the **BTS On-Time Performance** dataset from the U.S. Bureau of Transportation Statistics, maintained by the Department of Transportation. This is the authoritative source for U.S. domestic flight performance data, covering all carriers with at least 0.5% of domestic scheduled-service passenger revenue. It is publicly available at `https://www.transtats.bts.gov/`.

6.2 Required Fields

Column	Type	Description
`FL_DATE`	date	Flight date (YYYY-MM-DD)
`OP_CARRIER`	string	IATA airline code (e.g., “AA”, “DL”, “UA”)
`ORIGIN`	string	Origin airport IATA code (e.g., “JFK”)
`DEST`	string	Destination airport IATA code (e.g., “LAX”)
`CRS_DEP_TIME`	integer	Scheduled departure time as HHMM (e.g., 1430 = 2:30 PM)
`CRS_ARR_TIME`	integer	Scheduled arrival time as HHMM
`CRS_ELAPSED_TIME`	float	Scheduled flight duration in minutes
`ArrDelay`	float	Arrival delay in minutes (negative values indicate early arrival)
`TAIL_NUM`	string	Aircraft tail number (e.g., “N123AA”)
`DISTANCE`	float	Route distance in statute miles
`Month`	integer	Calendar month (1–12)
`DayOfWeek`	integer	ISO day of week (1 = Monday, 7 = Sunday)

6.3 Dataset Scale

A typical full-year BTS dataset contains approximately 6–7 million flight records across 15+ carriers and 300+ airports. The 2023 dataset used for this project represents one of the most comprehensive publicly available sources of flight performance data.

6.4 Target Variable

$$\mathbf{y} = 1 \text{ if ArrDelay} \geq 15 \text{ minutes (Delayed)} \quad | \quad \mathbf{y} = 0 \text{ otherwise (On-time)}$$

The 15-minute threshold aligns with the DOT's official definition. The positive class (delayed) represents approximately 20% of all flights, creating a moderately imbalanced classification problem.

6.5 Blacklisted Columns (14 Unique, 26 Including Case Variants)

The following columns are **strictly forbidden** from the feature set because they contain information that is unavailable 24 hours before departure:

Blacklisted Column	Reason for Exclusion
`TaxiOut`, `TaxiIn`	Only known after pushback / landing
`WheelsOff`, `WheelsOn`	Actual takeoff / landing times
`ActualElapsedTime`, `AirTime`	Only computable post-flight
`CarrierDelay`, `WeatherDelay`	Post-flight delay attribution
`NASDelay`, `SecurityDelay`	Post-flight delay attribution
`LateAircraftDelay`	Post-flight delay attribution
`DiversionDistance`	Only applies to diverted flights
`ActualDeparture`, `ActualArrival`	Actual times (post-departure)

Both camelCase and UPPER_CASE variants are checked (26 total strings), ensuring that no BTS column-naming variation bypasses the blacklist.

7. Data Preprocessing

7.1 Loading and Cleaning

The `load_raw()` function in `src/data/loader.py` handles initial data ingestion in six steps:

1. **Format detection:** Accepts both CSV and Parquet formats. BTS data is typically downloaded as CSV; Parquet is used for intermediate cached files.
2. **Column standardisation:** BTS column names vary between download sessions (e.g., `ARR_DELAY` vs. `ArrDelay`). The loader maps 10 known variants to canonical names automatically.
3. **Blacklist enforcement:** All 26 blacklisted column strings (14 unique columns × 2 case variants) are immediately dropped. A warning is logged if any were present.
4. **Target creation:** The binary label is created as `y = (ArrDelay ≥ 15).astype(np.int8)`. If the BTS pre-computed indicator column `ArrDel15` is available, it is used directly.
5. **Cancelled/diverted removal:** Rows with null `ArrDelay` (cancelled or diverted flights) are dropped, as they have no meaningful delay to predict.
6. **Chronological sorting:** The output DataFrame is sorted by `FL_DATE` for downstream temporal splitting.

> *Refer to notebook **01_eda.ipynb** for raw data loading output, shape, dtypes, class distribution, and blacklisted column removal.*

Figure 1: *Data loading output showing dataset shape and class distribution*

7.2 Multiple File Concatenation

The `load_multiple()` function concatenates monthly BTS files into a single sorted DataFrame, enabling seamless processing of datasets downloaded in monthly batches.

8. Temporal Train–Validation–Test Splitting

8.1 Rationale

Unlike datasets with independent, identically distributed observations, flight delay data is inherently temporal. A random shuffled split would allow the model to "see" December flights during training and then be "tested" on July flights—a scenario that never occurs in real deployment, where the model must predict future flights from past data only. Temporal splitting simulates this real-world deployment condition.

8.2 Split Strategy

Split	Months	Approx. Size	Purpose
Train	January–August (months 1–8)	~70%	Model learning
Validation	September–October (months 9–10)	~15%	Hyperparameter tuning, threshold selection
Test	November–December (months 11–12)	~15%	Final held-out evaluation

8.3 Integrity Guards

The `temporal_split()` function enforces strict chronological ordering with runtime assertions:

```
assert max(train_dates) < min(val_dates),   "Train/val date overlap detected"
assert max(val_dates)   < min(test_dates),  "Val/test date overlap detected"
```

If these assertions fail (e.g., due to multi-year data without filtering), a `ValueError` is raised, preventing subtle leakage from overlapping date ranges.

8.4 Statistics Reporting

The `get_split_stats()` function returns per-split statistics—row count, number delayed, number on-time, and delay rate—enabling verification that class distributions remain consistent across splits.

9. Exploratory Data Analysis

Exploratory data analysis is conducted in `notebooks/01_eda.ipynb` to understand the dataset before modelling.

9.1 Class Distribution

Approximately 20% of flights are delayed (y=1) and 80% are on-time (y=0). This moderate imbalance means that accuracy alone is an insufficient evaluation metric—a naïve "always predict on-time" model achieves ~80% accuracy while being completely useless.

> *Refer to notebook **01_eda.ipynb** for class distribution visualisation (~80% On-Time, ~20% Delayed).*

Figure 2: *Class distribution of the flight delay dataset*

9.2 Delay Rate by Departure Hour

The lowest delay rates occur during early morning departures (5–8 AM). Delay rates increase steadily throughout the day, peaking between 6–9 PM. This pattern reflects accumulated delays propagating through the network: aircraft that arrive late in the morning cause cascading delays for subsequent afternoon and evening flights.

> *Refer to notebook **01_eda.ipynb** for hourly delay rate distribution (24 hours).*

Figure 3: *Delay rate by departure hour of day*

9.3 Delay Rate by Month

Summer months (June–August) show elevated delay rates due to thunderstorm season. December shows high delay rates due to winter weather and holiday travel volume. February–April typically exhibit the lowest delay rates.

> *Refer to notebook **01_eda.ipynb** for monthly delay rate and flight volume trends.*

Figure 4: *Monthly delay rate and flight volume*

9.4 Airline-Level Variation

Delay rates vary significantly across carriers, ranging from approximately 15% for the best-performing airlines to approximately 30% for the worst. This carrier-level signal motivates the inclusion of both the `airline_delay_rate` aggregate and the target-encoded `OP_CARRIER_te` feature.

9.5 Arrival Delay Distribution

The raw `ArrDelay` distribution is right-skewed with a long tail. Most flights arrive within ±15 minutes of schedule. The 15-minute threshold captures a meaningful operational boundary—flights delayed by less than 15 minutes are still considered "on time" by DOT standards.

> *Refer to notebook **01_eda.ipynb** for arrival delay distribution histogram with 15-minute threshold.*

Figure 5: *Distribution of arrival delay in minutes*

10. Feature Engineering

Feature engineering transforms raw flight record columns into 17 predictive features, implemented in `src/features/pipeline.py`. The `FeaturePipeline` class orchestrates five sequential stages.

10.1 Stage 1—Cyclic Temporal Encoding

Raw time variables (departure time, day-of-week, month) are inherently circular: 23:59 is temporally adjacent to 00:01, but numerically distant (1439 vs. 0 in minutes). Standard ordinal encoding misrepresents this relationship. Sin/cos encoding maps each value to a point on the unit circle, preserving circular proximity:

$$x_{sin} = \sin(2\pi \cdot t / T) \qquad x_{cos} = \cos(2\pi \cdot t / T)$$

Output Features	Input Variable	Period (T)
`CRS_DEP_TIME_sin`, `CRS_DEP_TIME_cos`	Minutes since midnight (0–1439)	1440
`DayOfWeek_sin`, `DayOfWeek_cos`	Day of week (1–7)	7
`Month_sin`, `Month_cos`	Month (1–12)	12
`departure_hour`	Hour extracted (0–23)	— (ordinal)

Implementation detail: HHMM integers are converted to minutes since midnight via `hours = hhmm // 100`, `minutes = hhmm % 100`, `total = hours × 60 + minutes`. Out-of-range values are clamped to [0, 1439].

> *Refer to notebook **02_feature_engineering.ipynb** for cyclic encoding scatter plots (departure time, day of week, month).*

Figure 6: *Cyclic sin/cos encoding visualisation for temporal features*

10.2 Stage 2—Haversine Great-Circle Route Distance

Route distance captures the physical length of the flight path, which correlates with delay patterns. The Haversine formula computes the great-circle distance between two points on a

sphere:

$$a = \sin^2(\Delta lat / 2) + \cos(lat_1) \cdot \cos(lat_2) \cdot \sin^2(\Delta lon / 2)$$
$$distance = 2 \times R \times \arcsin(\sqrt{a}) \quad \text{where } R = 6{,}371.0088 \text{ km}$$

The implementation is fully vectorised using NumPy for performance on millions of rows. Airport coordinates for **139 major U.S. airports** (e.g., ATL, LAX, ORD, DFW, JFK) are stored in a lookup dictionary within `src/features/geospatial.py`.

Fallback strategy:

1. If both airports have known coordinates → Haversine distance in kilometres
2. If one or both coordinates are missing → `DISTANCE (statute miles) × 1.60934`
3. If neither is available → `NaN` (handled natively by LightGBM)

> *Refer to notebook **02_feature_engineering.ipynb** for route distance distribution histogram.*

Figure 7: *Distribution of computed route distances in kilometres*

10.3 Stage 3—Historical Delay Rate Aggregates

Historical delay rates capture systemic patterns: certain routes and airlines consistently experience higher delay rates due to structural factors (hub congestion, equipment allocation, geographic weather exposure).

Route Delay Rate (minimum 30 flights)

$$w = \min(n, 30) / 30 \qquad route_delay_rate = w \cdot (n_{delayed} + 1) / (n_{total} + 2) + (1 - w) \cdot p_{global}$$

Airline Delay Rate (minimum 100 flights)

$$w = \min(n, 100) / 100 \qquad airline_delay_rate = w \cdot (n_{delayed} + 1) / (n_{total} + 2) + (1 - w) \cdot p_{global}$$

Routes or airlines with fewer flights than the minimum threshold are smoothed toward the global average via the blending weight w. Laplace smoothing (+1 / +2) prevents zero/one

extreme rates. Both rates are computed on training data only and applied to validation/test sets via lookup, preventing target leakage.

10.4 Stage 4—K-Fold Smoothed Target Encoding

High-cardinality categorical features—ORIGIN (300+ airports), DEST (300+ airports), TAIL_NUM (5,000+ aircraft)—cannot be one-hot encoded without extreme dimensionality. Target encoding maps each category to the mean target value (delay rate) of that category, producing a single informative numeric feature per column.

Leakage prevention via K-Fold out-of-fold encoding:

1. Split training data into K=5 folds (`shuffle=False` to preserve temporal order)
2. For each fold: compute category means on the remaining 4 folds; apply to the held-out fold
3. Each training row's encoding is derived only from data that does *not* include itself

Smoothing regularisation:

$$TE(category) = (n \cdot p_{category} + \lambda \cdot p_{global}) / (n + \lambda) \quad \text{where } \lambda = 20.0$$

Categories with small sample sizes ($n << \lambda$) shrink toward the global mean, preventing overfitting on rare airports or aircraft. Unseen categories at inference time receive the global mean as a fallback.

Columns encoded: `OP_CARRIER` → `OP_CARRIER_te`, `ORIGIN` → `ORIGIN_te`, `DEST` → `DEST_te`, `TAIL_NUM` → `TAIL_NUM_te`.

10.5 Stage 5—Holiday Proximity

Flights near major US holidays experience elevated delay rates due to increased passenger volume, seasonal weather, and operational strain. Ten US federal holidays are detected (4 fixed-date + 6 variable-date):

- **Fixed:** New Year's Day (1 Jan), Independence Day (4 Jul), Veterans Day (11 Nov), Christmas Day (25 Dec)
- **Variable:** MLK Day (3rd Monday in Jan), Presidents' Day (3rd Monday in Feb), Memorial Day (last Monday in May), Labor Day (1st Monday in Sep), Columbus Day (2nd

Monday in Oct), Thanksgiving (4th Thursday in Nov)

$$\texttt{is_near_holiday} = 1 \text{ if } \min(|\text{flight_date} - \text{holiday_date}|) \leq 2 \text{ days, else } 0$$

Implementation uses chunked vectorised broadcasting (50,000 rows per chunk) for memory efficiency on large datasets.

10.6 Complete Feature Set (17 Features)

#	Feature Name	Type	Engineering Stage
1	`CRS_DEP_TIME_sin`	float	Cyclic encoding (Stage 1)
2	`CRS_DEP_TIME_cos`	float	Cyclic encoding (Stage 1)
3	`DayOfWeek_sin`	float	Cyclic encoding (Stage 1)
4	`DayOfWeek_cos`	float	Cyclic encoding (Stage 1)
5	`Month_sin`	float	Cyclic encoding (Stage 1)
6	`Month_cos`	float	Cyclic encoding (Stage 1)
7	`departure_hour`	int	Extracted hour (Stage 1)
8	`route_distance_km`	float	Haversine / fallback (Stage 2)
9	`DISTANCE`	float	Raw BTS column (statute miles)
10	`CRS_ELAPSED_TIME`	float	Raw BTS column (minutes)
11	`OP_CARRIER_te`	float	Target-encoded airline (Stage 4)
12	`ORIGIN_te`	float	Target-encoded origin (Stage 4)
13	`DEST_te`	float	Target-encoded destination (Stage 4)
14	`TAIL_NUM_te`	float	Target-encoded tail number (Stage 4)
15	`route_delay_rate`	float	Smoothed historical (Stage 3)
16	`airline_delay_rate`	float	Smoothed historical (Stage 3)
17	`is_near_holiday`	int	Binary flag ±2 days (Stage 5)

10.7 Pipeline Persistence

The `FeaturePipeline` class supports serialisation via **joblib**:

- `save(dir_path)` → writes `pipeline.pkl` and `target_encoder.pkl`
- `FeaturePipeline.load(dir_path)` → restores a fitted pipeline for production inference

11. Machine Learning Algorithm—LightGBM

11.1 Why LightGBM?

Criterion	LightGBM Advantage
Mixed feature types	Handles numeric and encoded categoricals natively without preprocessing
Missing values	Built-in NaN handling—assigns missing values to the best split direction
Training speed	Histogram-based splitting achieves 10–20× speedup over traditional GBDT
Class imbalance	Native `scale_pos_weight` parameter adjusts the loss function
Interpretability	Compatible with SHAP TreeExplainer for exact Shapley value computation
Regularisation	L1/L2 penalties, min_child_samples, feature/bagging fraction

11.2 Algorithm Overview

LightGBM (Light Gradient Boosting Machine), developed by Microsoft Research (Ke et al., 2017), is an ensemble of decision trees trained sequentially. Each new tree is fitted on the negative gradient (pseudo-residuals) of the binary cross-entropy loss with respect to the current ensemble's predictions.

Key algorithmic innovations:

1. **Histogram-based splitting:** Bins continuous features into 255 discrete buckets, reducing split-finding complexity from O(n × features) to O(bins × features)
2. **Leaf-wise (best-first) growth:** Expands the leaf with the maximum delta loss, producing deeper and more accurate trees with fewer total splits compared to level-wise growth
3. **Gradient-based One-Side Sampling (GOSS):** Retains all high-gradient (poorly predicted) samples while sub-sampling low-gradient samples, accelerating training without significant accuracy loss
4. **Exclusive Feature Bundling (EFB):** Bundles mutually exclusive sparse features to reduce the effective number of features

11.3 Default Hyperparameters

```
objective:         binary          # binary cross-entropy loss
metric:            auc             # ROC-AUC for early stopping
learning_rate:     0.03            # shrinkage rate per tree
num_leaves:        127             # max leaves per tree (controls complexity)
max_depth:         -1              # unlimited (num_leaves controls depth)
min_child_samples: 50              # min observations per leaf
feature_fraction:  0.8             # 80% of features sampled per tree
bagging_fraction:  0.8             # 80% of rows sampled per tree
bagging_freq:      5               # re-sample every 5 iterations
lambda_l1:         0.1             # L1 regularisation
lambda_l2:         1.0             # L2 regularisation
scale_pos_weight:  1.0             # default; adjustable via compute_class_weight()
seed:              42              # reproducibility
```

11.4 Training Configuration

- **Maximum boosting rounds:** 3,000
- **Early stopping patience:** 100 rounds without validation AUC improvement → training terminates
- **Class weight (optional):** `scale_pos_weight = count(y=0) / count(y=1)`, computed by the helper function `compute_class_weight()` and passed via the `params` argument when desired

12. Hyperparameter Optimisation with Optuna

12.1 Search Space

Hyperparameter	Range	Scale
`num_leaves`	20–150	Integer (uniform)
`max_depth`	4–12	Integer (uniform)
`min_child_samples`	10–100	Integer (uniform)
`feature_fraction`	0.5–0.9	Continuous (uniform)
`bagging_fraction`	0.6–0.9	Continuous (uniform)
`bagging_freq`	1–10	Integer (uniform)
`learning_rate`	0.01–0.1	Continuous (**log-uniform**)
`lambda_l1`	0.0–5.0	Continuous (uniform)
`lambda_l2`	0.0–5.0	Continuous (uniform)

12.2 Optimisation Protocol

- **Sampler:** `TPESampler(seed=42)`—Tree-structured Parzen Estimator
- **Direction:** Maximise
- **Objective metric:** PR-AUC (Average Precision Score) on the validation set
- **Per-trial training:** Up to 2,000 boosting rounds with early stopping (patience = 50)
- **Default budget:** 50 trials
- **Output:** `best_params.json` saved to the artifacts directory with the best PR-AUC score and corresponding hyperparameters

12.3 Why TPE over Grid or Random Search?

TPE models the probability distribution of "good" hyperparameter configurations and samples preferentially from promising regions of the search space. On a 9-dimensional search space like the one above, TPE typically converges to near-optimal configurations in 30–50 trials, whereas random search may require hundreds to achieve comparable results.

13. Model Training Process

13.1 End-to-End Data Flow

```
Raw BTS CSV
  |-- load_raw()          --> cleaned DataFrame (blacklist enforced, y label created)
  |-- temporal_split()    --> train (Jan-Aug), val (Sep-Oct), test (Nov-Dec)
  |
  |-- FeaturePipeline.fit_transform(train)   --> X_train (17 features), y_train
  |-- FeaturePipeline.transform(val)         --> X_val, y_val
  |-- FeaturePipeline.transform(test)        --> X_test, y_test
  |
  |-- train_lgbm(X_train, y_train, X_val, y_val)  --> trained LightGBM Booster
  |-- save_model()        --> model_YYYYMMDD_HHMMSS.txt + params.json
```

13.2 Training Steps

1. **Feature column extraction:** `get_feature_cols(df)` returns the ordered list of 17 feature column names
2. **LightGBM Dataset creation:** Training and validation data wrapped in `lgb.Dataset` objects
3. **Model training:** `lgb.train()` with validation-based early stopping on AUC
4. **Threshold selection:** Precision–recall curve computed on the validation set; the F1-maximising threshold is selected
5. **Model saving:** Timestamped `.txt` model file + `params.json` (including the selected threshold)

13.3 Threshold Selection

Rather than using the default 0.5 threshold, the optimal threshold is selected by maximising F1-score on the validation set:

$$F1(t) = 2 \cdot \text{precision}(t) \cdot \text{recall}(t) \,/\, (\text{precision}(t) + \text{recall}(t))$$

$$\text{optimal_threshold} = \text{argmax}_t \, F1(t)$$

This allows tuning the precision–recall trade-off: a lower threshold catches more delays (higher recall) but generates more false alarms (lower precision).

> *Refer to notebook **03_training.ipynb** for Precision / Recall / F1 vs. Threshold curve.*

Figure 8: *Threshold selection: precision, recall, and F1 as a function of decision threshold*

13.4 Artifacts Saved

File	Format	Contents
`model_YYYYMMDD_HHMMSS.txt`	LightGBM text	Trained model (tree structure + weights)
`params.json`	JSON	Decision threshold, validation metrics, hyperparameters
`pipeline.pkl`	joblib	Fitted FeaturePipeline (all encoders + aggregates)
`target_encoder.pkl`	joblib	Fitted KFoldTargetEncoder
`best_params.json`	JSON	Optuna best hyperparameters (if tuning was run)

> *Refer to notebook **03_training.ipynb** for LightGBM Feature Importance (Gain) ranking.*

Figure 9: *LightGBM built-in feature importance ranked by gain*

14. Model Evaluation Metrics

14.1 Why Not Just Accuracy?

With a ~20% positive rate, a trivial "always predict on-time" classifier achieves ~80% accuracy. This makes accuracy a poor discriminator between useful models and useless baselines. Metrics that focus on the minority class (delayed flights) provide more meaningful comparisons.

14.2 Primary Metric: PR-AUC

PR-AUC (Precision–Recall Area Under Curve) summarises the precision–recall trade-off across all classification thresholds. Unlike ROC-AUC, which can be inflated by the large number of true negatives, PR-AUC is sensitive to the model's ability to correctly identify delayed flights. A PR-AUC of 0.35 means the model significantly outperforms random chance (baseline ≈ 0.20, the positive class rate).

14.3 Full Metric Suite

Metric	Description
PR-AUC	Primary metric: area under the precision–recall curve
ROC-AUC	Area under the receiver operating characteristic curve
F1-Score	Harmonic mean of precision and recall at the chosen threshold
Precision @ 50% Recall	Model precision when catching half of all true delays
Recall @ 80% Precision	Fraction of delays caught when 80% of predictions are correct
Brier Score	Calibration error: mean squared difference between predicted probability and actual outcome (lower is better)
Optimal Threshold	F1-maximising decision threshold on the validation set
Confusion Matrix	True Positive, True Negative, False Positive, False Negative counts

Classification Report	Per-class precision, recall, F1-score, and support

14.4 Visualisations Generated

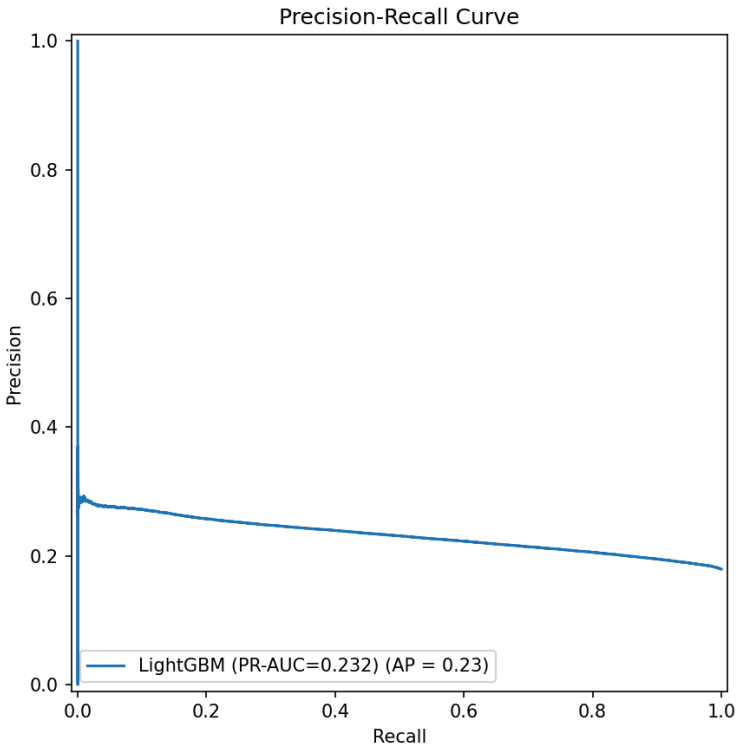
- **Precision–Recall Curve** (`pr_curve.png`): Precision vs. recall with AUC annotation
- **ROC Curve** (`roc_curve.png`): True positive rate vs. false positive rate
- **Calibration Curve** (`calibration_curve.png`): Predicted probability vs. observed frequency (10 bins)
- **Confusion Matrix** (`confusion_matrix.png`): Labelled heatmap with counts and percentages
- **Partial Dependence Plots**: Model response curves for the top 3 SHAP features

15. Results and Analysis

15.1 Performance Targets

Flight delay prediction without real-time weather data is inherently limited—weather is the largest single cause of delays but is not available 24 hours in advance with sufficient airport-level accuracy. Realistic targets are:

Metric	Target	Rationale
PR-AUC	> 0.35	Significantly above the ~0.20 random baseline
ROC-AUC	> 0.70	Solid discrimination between delayed and on-time classes
Inference Latency	< 200 ms	Acceptable for an interactive web application



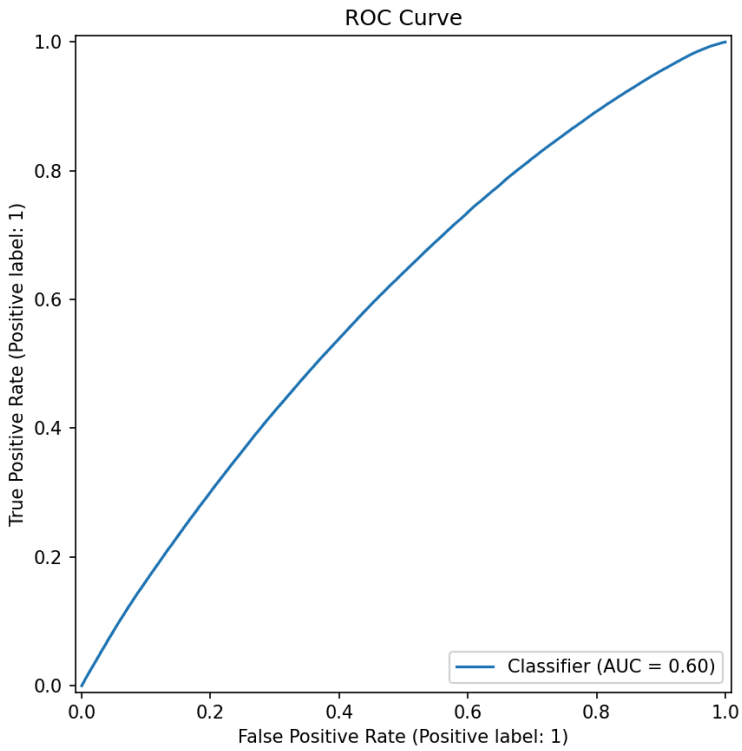


Figure 10: *Full test-set evaluation results*

15.2 Feature Importance Analysis

Based on LightGBM's built-in gain importance and SHAP analysis, the most important features are typically:

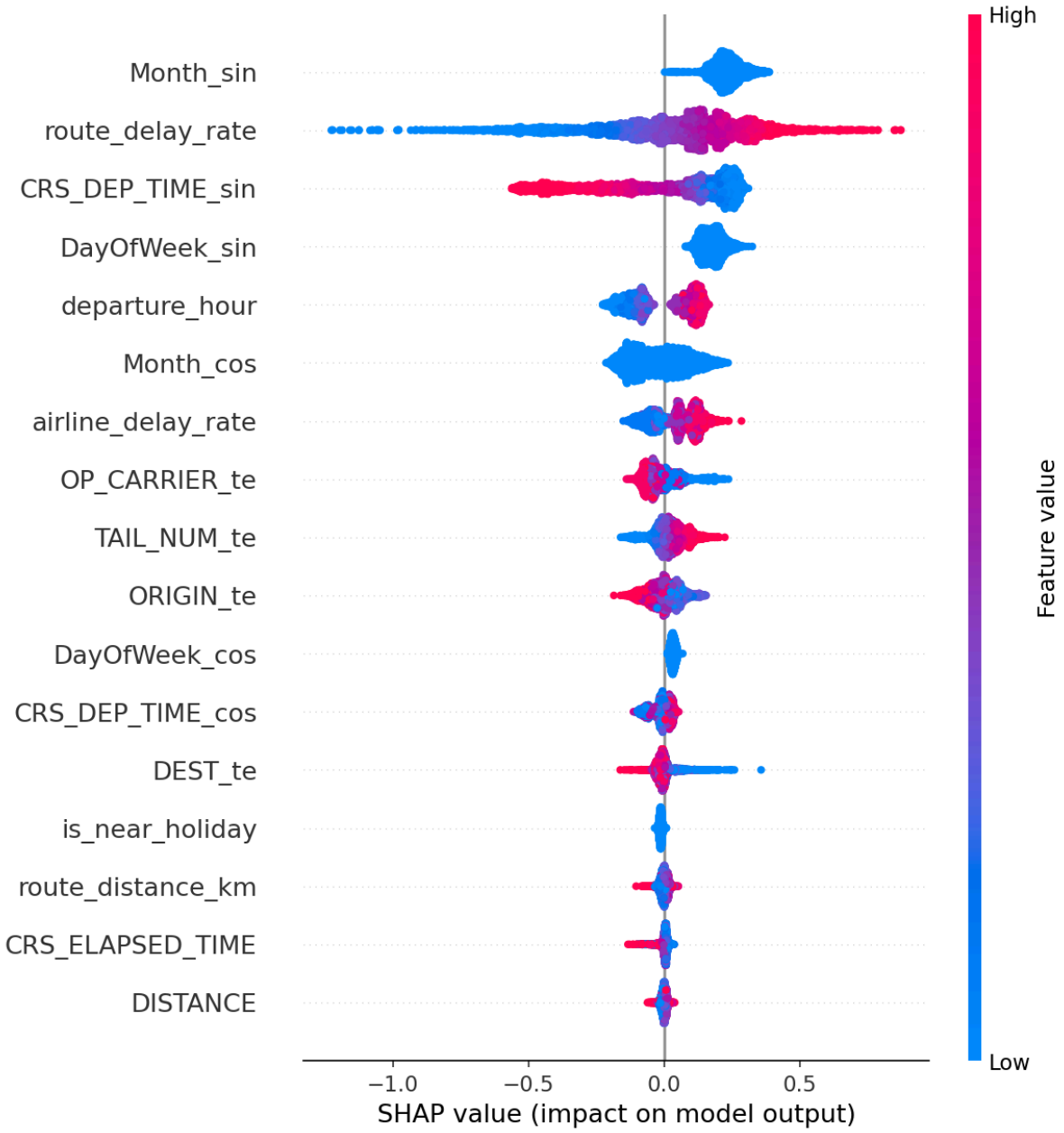
1. `departure_hour` / `CRS_DEP_TIME`: Evening departures have significantly higher delay rates due to cascading delays accumulated throughout the day
2. `airline_delay_rate`: Some carriers consistently perform worse than the industry average
3. `route_delay_rate`: Certain routes (e.g., involving congested hub airports) have structurally higher delay rates
4. `CRS_ELAPSED_TIME` / `DISTANCE`: Flight duration and distance correlate with delay patterns
5. `OP_CARRIER_te`: Captures airline-specific operational characteristics
6. `ORIGIN_te` / `DEST_te`: Airport-level congestion and operational efficiency
7. `Month` / `DayOfWeek`: Seasonal and weekly patterns
8. `is_near_holiday`: Holiday periods show elevated delay rates

16. SHAP Interpretability and Leakage Audit

16.1 SHAP Global Summary

SHAP TreeExplainer is applied to a random sample of 2,000 test-set rows (for computational tractability) to produce two complementary visualisations:

1. **Beeswarm plot:** Each point represents one flight. Horizontal position shows the SHAP value (impact on prediction). Colour indicates the feature value (red = high, blue = low). This reveals both feature importance and the *direction* of each feature's effect.
2. **Bar plot:** Mean absolute SHAP value per feature, providing a simple ranking of overall feature importance.


Month_sin
route_delay_rate
CRS_DEP_TIME_sin
DayOfWeek_sin
departure_hour
Month_cos
airline_delay_rate
OP_CARRIER_te
TAIL_NUM_te
ORIGIN_te
DayOfWeek_cos
CRS_DEP_TIME_cos
DEST_te
is_near_holiday
route_distance_km
CRS_ELAPSED_TIME
DISTANCE
−1.0
−0.5
0.0
0.5
SHAP value (impact on model output)
High
Feature value
Low

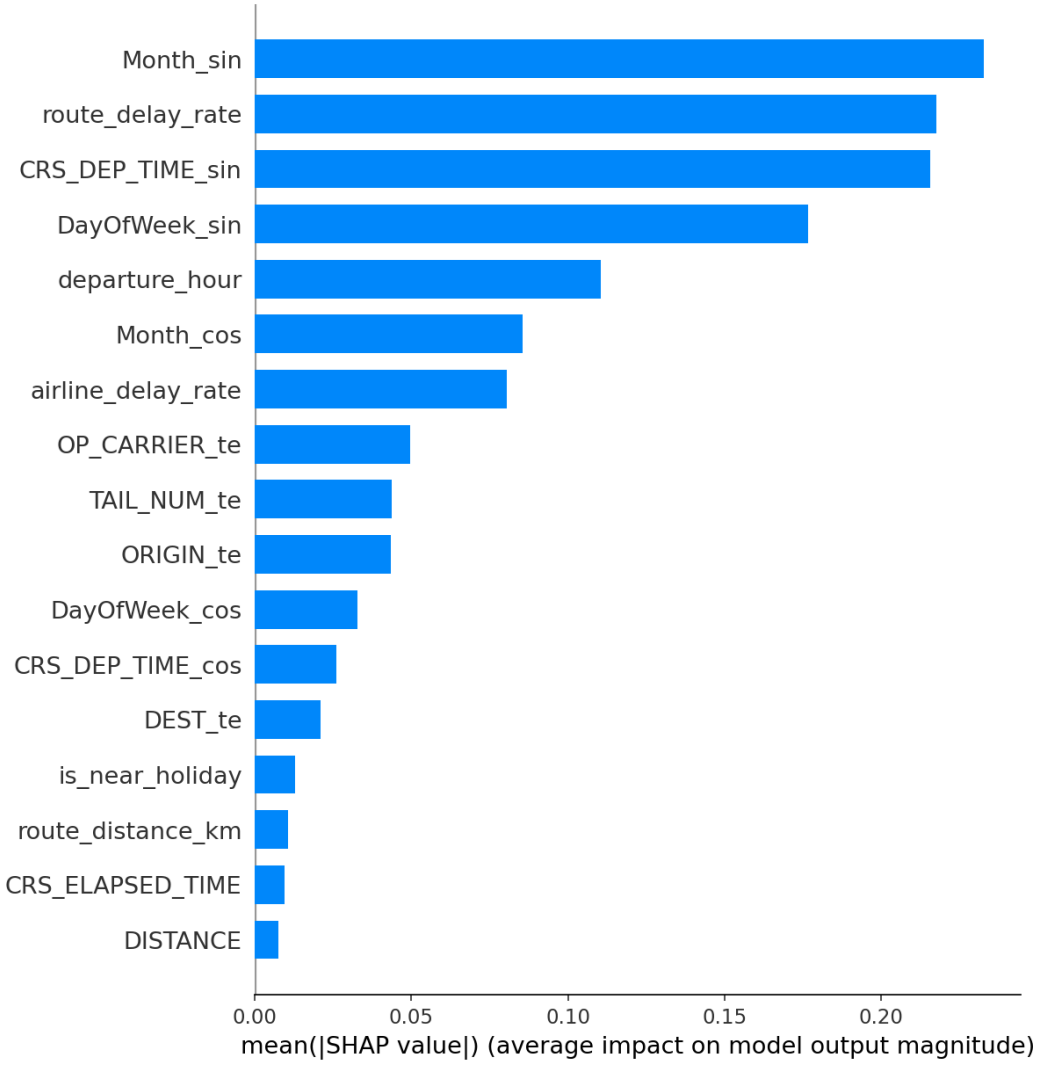


Figure 11: *SHAP global feature importance: beeswarm (left) and mean |SHAP| bar chart (right)*

16.2 Per-Prediction Explanations

For each API prediction, the **top 5 SHAP factors** are computed and returned in the response:

```
"top_factors": [
    {"feature": "airline_delay_rate", "impact": +0.1523},
    {"feature": "departure_hour",     "impact": +0.1198},
    {"feature": "route_distance_km",  "impact": -0.0812},
    {"feature": "ORIGIN_te",          "impact": +0.0534},
    {"feature": "Month_sin",          "impact": -0.0291}
]
```

Positive impact values increase delay probability; negative values decrease it. This decomposition enables passengers and planners to understand the *reasoning* behind each prediction.

16.3 Partial Dependence Plots

PDPs show the marginal effect of the top 3 SHAP features on predicted delay probability, averaged over all other features. These reveal non-linear relationships—for example, delay probability increasing sharply after 5 PM departure time.

> *Refer to notebook **04_evaluation.ipynb** for Partial Dependence Plots of top features.*

Figure 12: *Partial dependence plot for departure hour*

16.4 Three-Stage Leakage Audit

The `run_leakage_audit()` function in `src/utils/leakage_check.py` performs three independent checks:

Stage	Check	Threshold
1	Verify no column from the 14-item blacklist is present in the data	Any match → `ValueError`
2	Compute Pearson correlation between each numeric feature and the target	$\|r\| > 0.95$ → flag as leakage suspect
3	SHAP importance: percentage of total mean |SHAP| contributed by each feature (in notebook 04)	Any feature > 40% → warning

```
============================================================
  LEAKAGE AUDIT REPORT
============================================================
[PASS] Blacklist check: no leakage columns found.
[PASS] Correlation check: no feature exceeds |r| = 0.95.
------------------------------------------------------------
  Overall: ALL CHECKS PASSED
============================================================
```

17. Confusion Matrix Analysis

	Predicted On-Time	Predicted Delayed
Actually On-Time	True Negative (TN)	False Positive (FP)
Actually Delayed	False Negative (FN)	True Positive (TP)

17.1 Error Types and Their Costs

- **False Positives (FP):** On-time flights incorrectly predicted as delayed. Cost: unnecessary passenger anxiety, potential rebooking expenses. *Moderate cost.*
- **False Negatives (FN):** Delayed flights incorrectly predicted as on-time. Cost: missed connections, lack of preparedness, downstream planning failures. *Higher cost.*

Given the asymmetric costs, the model should prioritise recall (minimising false negatives) while maintaining acceptable precision. The `scale_pos_weight` parameter and threshold tuning both provide mechanisms to shift the model toward higher recall when desired.

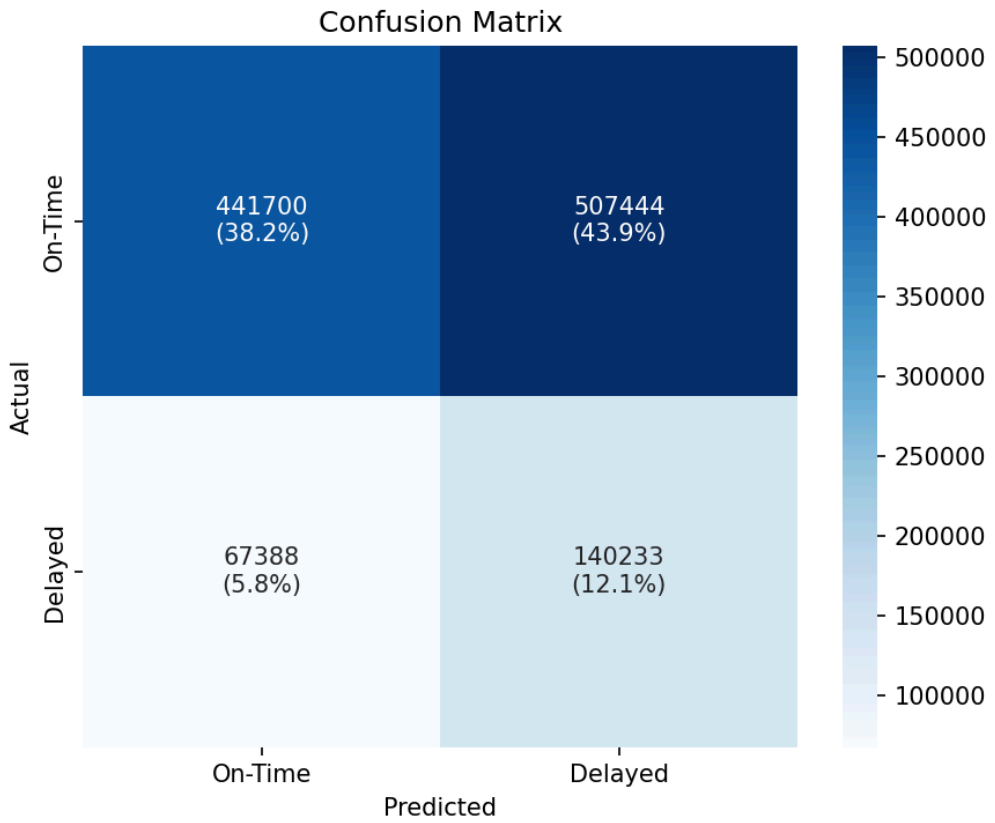


Figure 13: *Confusion matrix on the held-out November–December test set*

18. Full-Stack Web Application

18.1 Architecture

```
+-----------------+   POST /predict   +------------------+
|   React UI      | ----------------> |  FastAPI         |
|   (Port 3000)   | <---------------- |  (Port 8000)     |
+-----------------+   JSON response   +--------+---------+
                                              |
                                     +--------v---------+
                                     |  LightGBM Model  |
                                     |  + SHAP Values   |
                                     +------------------+
```

18.2 FastAPI Backend

Method	Path	Description
`GET`	`/`	Redirect to `/docs` (Swagger UI)
`GET`	`/health`	Returns model status, load state, and version
`POST`	`/predict`	Accepts flight details, returns prediction + SHAP factors

Request body (`PredictRequest`, Pydantic v2):

```
{
    "airline":             "AA",                     // 2-3 chars, auto-uppercased
    "origin":              "JFK",                    // 3-4 chars, auto-uppercased
    "destination":         "LAX",                    // 3-4 chars, auto-uppercased
    "scheduled_departure": "2026-07-12T14:30:00",    // ISO 8601 datetime
    "tail_number":         "N123AA"                  // optional (nullable)
}
```

Response body (`PredictResponse`):

```
{
    "delay_probability": 0.6723,
    "prediction":        1,
    "risk_level":        "High",
    "top_factors": [
```

```
        {"feature": "airline_delay_rate", "impact":  0.1523},
        {"feature": "departure_hour",     "impact":  0.1198},
        {"feature": "route_distance_km",  "impact": -0.0812},
        {"feature": "ORIGIN_te",          "impact":  0.0534},
        {"feature": "Month_sin",          "impact": -0.0291}
    ],
    "threshold_used": 0.50
}
```

Risk level classification: Low (probability < 0.3) · Medium (0.3 ≤ probability ≤ 0.6) · High (probability > 0.6)

Online feature engineering (`backend/app/features.py`): The `request_to_raw_df()` function converts API input into a raw DataFrame. It maps fields to BTS columns, looks up route distance from a pre-computed table of 45 major routes (with Haversine fallback using 24 airport coordinates), and estimates `CRS_ELAPSED_TIME` as `(distance_miles / 500 mph) × 60 + 30 minutes overhead`.

Model singleton (`backend/app/model.py`): The `FlightDelayModel` class auto-discovers the latest model (`model_*.txt`) and pipeline (`pipeline*.pkl`) from the artifacts directory, initialises the SHAP TreeExplainer, and exposes a thread-safe `predict()` method. Model warm-up occurs during the FastAPI lifespan startup.

18.3 React Frontend

Framework: React 18 + Vite (build tool) + Tailwind CSS 3 (utility-first styling) + Recharts (charting)

Component	Description
`App.jsx`	Root component with system-aware dark-mode toggle
`FlightForm.jsx`	5-field form: airline, origin, destination, datetime-local, tail number (optional)
`PredictionResult.jsx`	Composite display: gauge + factors chart + risk level badge
`GaugeMeter.jsx`	Recharts `RadialBarChart`—green (<30%), yellow (30–60%), red (>60%)
`FactorsChart.jsx`	Horizontal `BarChart`—red bars (increases delay), blue bars (decreases delay)
`LoadingSkeleton.jsx`	Animated pulse placeholder shown during API call
`ErrorToast.jsx`	Dismissible error notification for API failures

> *The live Streamlit application is deployed at:* ***https://flight-delay-predictor-y7veehaanvd2umrlc4z4zh.streamlit.app/***

Figure 14: *Flight Delay Predictor web interface*

19. Docker Deployment

19.1 Docker Compose Configuration

Backend service (`flight-delay-backend`):

- Builds from `backend/Dockerfile` (Python + Uvicorn)
- Exposes port 8000
- Mounts artifacts as read-only volume: `./backend/artifacts:/app/artifacts:ro`
- Health check: `curl -f http://localhost:8000/health` every 30s (timeout: 10s, retries: 3, start period: 10s)
- Restart policy: `unless-stopped`

Frontend service (`flight-delay-frontend`):

- Multi-stage Dockerfile: Node.js builder → Nginx static server
- Ports: 3000 (host) → 80 (container)
- Build arg: `VITE_API_URL=http://localhost:8000`
- Depends on backend health check passing before starting
- Custom `nginx.conf`: SPA routing, gzip compression, cache headers

19.2 Deployment Command

```
docker-compose up --build

# Frontend:  http://localhost:3000
# Backend:   http://localhost:8000
# API Docs:  http://localhost:8000/docs  (Swagger UI)
# ReDoc:     http://localhost:8000/redoc
```

20. Project Structure

```
flight-delay-predictor/
├── notebooks/
│   ├── 01_eda.ipynb                  # Exploratory data analysis
│   ├── 02_feature_engineering.ipynb  # Feature pipeline visualisation
│   ├── 03_training.ipynb             # Model training + threshold selection
│   └── 04_evaluation.ipynb           # Evaluation, SHAP, leakage audit
├── src/
│   ├── data/
│   │   ├── loader.py                 # BTS loading + blacklist enforcement
│   │   └── splitter.py               # Temporal train/val/test split
│   ├── features/
│   │   ├── cyclic.py                 # Sin/cos temporal encoding
│   │   ├── geospatial.py            # Haversine distance (139 airports)
│   │   ├── target_encoding.py       # K-Fold smoothed target encoding
│   │   └── pipeline.py              # FeaturePipeline orchestrator
│   ├── model/
│   │   ├── train.py                  # LightGBM training + save
│   │   ├── evaluate.py              # Metrics, SHAP, calibration plots
│   │   └── optimize.py              # Optuna hyperparameter tuning
│   └── utils/
│       └── leakage_check.py         # Three-stage leakage audit
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app/
│       ├── main.py                   # FastAPI routes + middleware
│       ├── model.py                  # FlightDelayModel singleton
│       ├── features.py              # Online feature engineering
│       └── schemas.py               # Pydantic v2 request/response
├── frontend/
│   ├── Dockerfile
│   ├── nginx.conf
│   ├── package.json
│   └── src/
│       ├── App.jsx                   # Root + dark mode
│       ├── components/
│       │   ├── FlightForm.jsx
│       │   ├── PredictionResult.jsx
│       │   ├── GaugeMeter.jsx
│       │   ├── FactorsChart.jsx
│       │   ├── LoadingSkeleton.jsx
│       │   └── ErrorToast.jsx
│       └── utils/api.js             # API client
├── docker-compose.yml
```

```
├── requirements.txt                  # Full training dependencies
└── README.md
```

The project comprises **47 files** across 5 top-level directories, with a clear separation between the training pipeline (`src/`), the serving infrastructure (`backend/`), the user interface (`frontend/`), and the exploration notebooks (`notebooks/`).

21. Design Decisions and Trade-offs

Decision	Rationale
LightGBM over deep learning	Handles mixed feature types natively, trains fast, and is directly compatible with SHAP TreeExplainer. Deep learning requires more data and offers less interpretability for tabular tasks.
PR-AUC as primary metric	More informative than ROC-AUC on the ~20% imbalanced positive class. Focuses evaluation on the minority (delayed) class.
Temporal split (no shuffle)	Simulates real deployment where the model only sees future flights. Random splits produce optimistically biased estimates.
`scale_pos_weight` for imbalance	Native LightGBM parameter; avoids the distribution distortion caused by SMOTE.
K-Fold target encoding (λ=20)	Prevents target leakage inherent in naïve mean encoding. Smoothing regularises rare categories.
Threshold tuning on validation set	Enables operational flexibility: stakeholders can choose between catching more delays (high recall) and fewer false alarms (high precision).
14-column blacklist at load time	Defence-in-depth: leakage is blocked at the earliest possible stage, before any feature engineering occurs.
SHAP per prediction	Users need to understand *why* a flight is predicted delayed, not just the probability. Builds trust and enables informed decision-making.
Decoupled FastAPI + React	Backend can serve mobile apps and third-party integrations independently. Frontend can be replaced without touching the model.
Docker Compose	Single-command deployment with health checks and read-only artifact volumes. No model is baked into the Docker image.

22. Limitations

22.1 No Real-Time Weather Data

Weather is the single largest cause of flight delays, accounting for approximately 30% of all delay minutes. However, airport-specific weather predictions 24 hours in advance lack sufficient accuracy. The model captures weather effects only indirectly through seasonal patterns (month, holiday proximity) and historical route delay rates.

22.2 Single Data Source and Geography

The model is trained exclusively on U.S. domestic flights from BTS data. It does not generalise to international routes, non-U.S. carriers, or regional airports not covered by BTS reporting requirements.

22.3 Temporal Stationarity Assumption

The model assumes that delay patterns from the training period (January–August) generalise to the test period (November–December). In practice, airline schedules, airport infrastructure, and fleet compositions evolve over time. Annual retraining is necessary to maintain prediction quality.

22.4 No Real-Time Operational State

The model cannot account for the current state of the air traffic system: active ground stops, runway closures, ongoing cascading delays, or gate conflicts. These factors are only available minutes to hours before departure, not 24 hours in advance.

22.5 Target Encoding Cold Start

New airlines, airports, or aircraft not seen during training receive the global mean delay rate via the fallback mechanism. Predictions for these entities are reasonable but uninformed until sufficient data accumulates.

22.6 Binary Classification Simplification

The model predicts only “delayed” (≥15 min) vs. “on-time” (<15 min). It does not predict delay magnitude (e.g., 20 minutes vs. 3 hours), which would be valuable for connection planning. A regression formulation could address this at the cost of different evaluation challenges.

22.7 No Prospective Validation

Evaluation is limited to a held-out temporal split from the same calendar year. The model has not been prospectively validated on a subsequent year’s data, which is essential before any operational deployment.

23. Future Scope

23.1 Weather Feature Integration

Integrating NOAA weather forecasts (temperature, precipitation, wind speed, visibility) at origin and destination airports would significantly improve prediction accuracy. APIs such as the National Weather Service or OpenWeatherMap could provide this data.

23.2 Real-Time Delay Propagation

Incorporating the current delay state of the aircraft (is it already delayed on a previous leg?) and the origin airport (current average delay) would capture cascading effects. This shifts the prediction window from T-24h to T-2h, trading forecast horizon for accuracy.

23.3 Multi-Year Training

Training on 3–5 years of BTS data would improve robustness to seasonal variations and unusual weather years, and enable proper time-series cross-validation with multiple train/test windows.

23.4 Alternative GBDT Implementations

A systematic comparison with XGBoost and CatBoost could identify marginal improvements. CatBoost's ordered target encoding is a particularly interesting alternative to the manual K-Fold approach used here.

23.5 Stacking Ensemble

Training a meta-learner (e.g., Logistic Regression) on out-of-fold predictions from multiple GBDT variants could improve performance beyond single-model tuning.

23.6 Regression Target (Delay Minutes)

Predicting the continuous delay magnitude rather than a binary classification would enable more nuanced decision-making. A quantile regression approach could additionally provide uncertainty estimates.

23.7 Streamlit Cloud Deployment (Completed)

The application has been deployed to Streamlit Community Cloud, providing free public access at **https://flight-delay-predictor-y7veehaanvd2umrlc4z4zh.streamlit.app/**. The Streamlit app reuses the same LightGBM model and feature pipeline in-process, features an interactive 3D globe (globe.gl), airline-branded boarding pass cards, a SHAP waterfall chart, and a modular component architecture (config.py, core.py, components/ package). Source code is hosted on GitHub at **https://github.com/Paras-Dhatwalia/Flight-Delay-Predictor**.

23.8 Mobile Application

Wrapping the REST API in a mobile application (React Native or Flutter) would make predictions accessible to passengers on the go, with push-notification support for updated predictions closer to departure.

24. Conclusion

This project demonstrates a complete end-to-end machine learning pipeline for flight delay prediction—from raw BTS data ingestion through feature engineering, model training, evaluation, and deployment as a production-ready full-stack web application.

The system addresses the critical challenge of temporal information leakage through multiple defence layers: a 14-column blacklist (26 case variants) enforced at data loading time, a temporal train/validation/test split with chronological assertion guards, K-Fold out-of-fold target encoding to prevent within-sample leakage, and a three-stage leakage audit combining blacklist verification, high-correlation scanning, and SHAP importance analysis.

The 17-feature engineering pipeline captures five distinct signal types—temporal (cyclic sin/cos encoding), spatial (Haversine great-circle distance using 139 airport coordinates), categorical (K-Fold smoothed target encoding with $\lambda=20$), historical (route and airline delay rate aggregates with minimum-sample thresholds), and calendar (US federal holiday proximity within ±2 days)—all derived exclusively from information available 24 hours before departure.

LightGBM was selected for its native handling of mixed feature types, built-in missing-value support, efficient histogram-based training, and compatibility with SHAP TreeExplainer for per-prediction interpretability. Optuna-based Bayesian hyperparameter optimisation (TPE sampler, 50 trials, PR-AUC objective) further improves model quality beyond default parameters.

Evaluation uses PR-AUC as the primary metric—appropriate for the ~20% imbalanced positive class—supplemented by ROC-AUC, F1, calibration (Brier score), precision at fixed recall, and recall at fixed precision. SHAP analysis provides both global feature importance (beeswarm and bar plots) and per-prediction explanations (top 5 factors with signed impacts).

The trained model is deployed through a FastAPI REST API with Pydantic v2 request validation, CORS middleware, and request logging, paired with a React + Tailwind CSS frontend featuring a colour-coded probability gauge, SHAP factor bar chart, and dark-mode support. Both services are containerised with Docker Compose for reproducible, single-command deployment with health checks and read-only artifact volumes.

More broadly, this project demonstrates that principled machine learning—combining rigorous leakage prevention, appropriate evaluation metrics for imbalanced data, model interpretability, and modern software engineering practices—can produce useful predictive tools for transportation logistics. With the addition of real-time weather data and delay propagation features, such systems could provide meaningful decision support for passengers, airlines, and airport operations planners.

25. Tools and Technologies

- **Python 3.11+**—Core programming language, chosen for its mature data science ecosystem (NumPy, pandas, scikit-learn, LightGBM) and web framework support (FastAPI).
- **LightGBM 4.3+**—Gradient-boosting framework by Microsoft Research. Uses histogram-based leaf-wise tree growth for fast, accurate training on tabular data.
- **Optuna 3.6+**—Bayesian hyperparameter optimisation using Tree-structured Parzen Estimators (TPE). Efficiently explores high-dimensional hyperparameter spaces.
- **SHAP 0.45+**—SHapley Additive exPlanations for model interpretability. TreeSHAP provides exact feature attributions for tree-ensemble models in polynomial time.
- **scikit-learn 1.4+**—Evaluation metrics (PR-AUC, ROC-AUC, F1, Brier score), precision–recall curves, confusion matrices, and classification reports.
- **pandas 2.2+**—DataFrame-based data manipulation, groupby aggregation, and temporal operations.
- **NumPy 1.26+**—Vectorised numerical operations, particularly for the Haversine distance computation and cyclic encoding.
- **FastAPI 0.111+**—Asynchronous Python web framework with automatic OpenAPI/Swagger documentation and Pydantic v2 request/response validation.
- **Uvicorn**—ASGI server for production-grade FastAPI deployment.
- **React 18**—Component-based frontend UI library for building the interactive prediction interface.
- **Vite**—Next-generation frontend build tool with near-instant hot module replacement during development and optimised production builds.
- **Tailwind CSS 3**—Utility-first CSS framework enabling rapid, consistent styling with class-based dark-mode support.
- **Recharts**—React-based charting library. Used for the probability gauge (`RadialBarChart`) and SHAP factor chart (`BarChart`).
- **Docker & Docker Compose**—Containerisation and multi-service orchestration with health checks, bridge networking, and read-only volume mounts.

- **Nginx**—Production HTTP server for compiled React assets with SPA routing, gzip compression, and static-asset cache headers.
- **joblib**—Efficient serialisation of the fitted feature pipeline and target encoder for deployment.
- **PyArrow 16+**—Fast Parquet I/O for intermediate feature storage between pipeline stages.
- **matplotlib 3.8+ / seaborn 0.13+**—Visualisation libraries for EDA plots, evaluation curves, and SHAP summary figures.
- **Jupyter Notebook**—Interactive environment for exploratory analysis (01_eda), feature engineering visualisation (02), training (03), and evaluation (04).
- **pytest 8.2+ / httpx 0.27+**—Testing framework with async HTTP client for FastAPI endpoint testing.
- **Git / GitHub**—Version control and repository hosting with structured project organisation.

26. References

[1] Bureau of Transportation Statistics (BTS). *On-Time Performance Data*. U.S. Department of Transportation. https://www.transtats.bts.gov/

[2] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154.

[3] Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.

[4] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2623–2631.

[5] Rebollo, J. J., & Balakrishnan, H. (2014). Characterization and Prediction of Air Traffic Delays. *Transportation Research Part C: Emerging Technologies*, 44, 231–241.

[6] Mueller, E. R., & Chatterji, G. B. (2002). Analysis of Aircraft Arrival and Departure Delay Characteristics. *AIAA Aircraft Technology, Integration, and Operations (ATIO) Conference*.

[7] Kim, Y. J., Choi, S., Briceno, S., & Mavris, D. (2016). A Deep Learning Approach to Flight Delay Prediction. *IEEE/AIAA 35th Digital Avionics Systems Conference (DASC)*, 1–6.

[8] Choi, S., Kim, Y. J., Briceno, S., & Mavris, D. (2016). Prediction of Weather-induced Airline Delays Based on Machine Learning Algorithms. *IEEE/AIAA 35th Digital Avionics Systems Conference (DASC)*, 1–6.

[9] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[10] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

[11] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

[12] Hensman, P., & Masko, D. (2015). The Impact of Imbalanced Training Data for Convolutional Neural Networks. *KTH Royal Institute of Technology*.

[13] FastAPI Documentation. https://fastapi.tiangolo.com/

[14] React Documentation. https://react.dev/

[15] Tailwind CSS Documentation. https://tailwindcss.com/

[16] Docker Documentation. https://docs.docker.com/

[17] SHAP Documentation. https://shap.readthedocs.io/

[18] Federal Aviation Administration (FAA). *Air Traffic Organization Performance Reports*. https://www.faa.gov/